nara-kosensai

システム構成・仕様書

対象リポジトリ: nara-kosensai/manifest(旧 config)、 fumikun/2026kosensai
最終更新: 2026-08-29

目次

1 概要 2
2 アーキテクチャ 2
2.1 リクエスト経路 2
2.2 namespace 構成 3
3 ネットワーク・セキュリティ 3
3.1 公開ポート 3
3.2 TLS / mTLS 3
3.3 RBAC 4
4 シークレット管理 4
5 CI/CD パイプライン 5
6 監視 5
7 障害と設計判断 5
7.1 nftables ルールのインターフェーススコープ(2026-08-29) 5
8 リファレンス 6

1 概要

nara-kosensai は、単一 VPS 上のシングルノード k3s クラスタで運用されている。旧構成(VPS ネイティブの nginx + 複数の docker-compose)から全面移行したもので、外部通信は原則 Cloudflare Tunnel 経由のみとし、 クラスタはノードの公開インターフェースに極力ポートをバインドしない。唯一の例外が本番アプリ (`nara-kosensai.com` / `www.nara-kosensai.com`)で、Cloudflare の Origin Authentication(mTLS)を 使い、Tunnel を経由せず直接オリジンに到達させる。

設定は 2 つのリポジトリで管理する。

リポジトリ	内容
`nara-kosensai/manifest`	クラスタブートストラップ(`k3s/`)、特定サービスに紐づかない共有プラットフォーム設定 (`k8s/`: cloudflared, Grafana/Loki/Prometheus, 過去年度アーカイブ, storybook, kosensai-form-preview)、この仕様書
`fumikun/2026kosensai`	アプリ本体(backend/frontend/db/pgadmin/website)の Dockerfile・k8s マニフェスト (`infra/k8s/base/`, `overlays/{prod,test}`)・CI/CD

2 アーキテクチャ

2.1 リクエスト経路

外部からのリクエストは、宛先ホストによって 2 つの異なる経路のいずれかを通る。両方とも最終的に 同一の Traefik Pod(namespace: `kube-system`)へ到達するが、Traefik の中では別々の entrypoint として扱われ、互いに干渉しない。

段階	本番 — mTLS Origin Auth	それ以外 — Cloudflare Tunnel
対象ホスト	`nara-kosensai.com` / `www.nara-kosensai.com`	`test.*` / `storybook.*` / `grafana.*` / `pgadmin*.*` / `info-check.*`
Cloudflare エッジ	SSL/TLS: Full (strict) + Authenticated Origin Pulls	Cloudflare Tunnel(通常の DNS/SSL モード設定とは独立に動作)
VPS への入口	公開インターフェース `ens3:443`(hostPort)	`cloudflared` Pod(namespace: `network`)、VPS 側からの outbound 接続のみ
フィルタ	nftables raw prerouting: `iifname == ens3` かつ `tcp dport 443` かつ `ip saddr ∉ Cloudflare 公開 IP レンジ` → drop(無応答)	該当なし(公開ポートを一切持たないため、外部からの直接到達手段自体が存在しない)
Traefik entrypoint	`websecure-mtls` ・hostPort 443 ・サーバ証明書: Cloudflare Origin CA 証明書 ・`TLSOption`(`RequireAndVerifyClientCert`) で Cloudflare の クライアント証明書を検証	`web` ・ClusterIP :80 のみ ・ホストへの bind なし

段階	本番 — mTLS Origin Auth	それ以外 — Cloudflare Tunnel
到達先 namespace	`prod`	`test` / `web-static` / `monitoring`

ポイント: `websecure-mtls` だけがホストポートに紐づいた entrypoint であり、`web` にはホストへの bind が一切ない。したがって `web` 側に残された IngressRoute は、たとえ nftables のフィルタや クライアント証明書の検証が何らかの理由で回避されたとしても、インターネットからは構造的に 到達不可能である(§6 障害と設計判断も参照)。

2.2 namespace 構成

namespace	代表ホスト	内容
`prod`	`nara-kosensai.com` / `www`	frontend・backend・db (StatefulSet)・website・pgadmin
`test`	`test.nara-kosensai.com`	frontend・backend・db・website・pgadmin
`web-static`	`storybook.*` / `info-check.*`	www-archive(過去年度アーカイブ)・storybook・form-preview
`monitoring`	`grafana.nara-kosensai.com`	grafana・loki・prometheus・node-exporter・kube-state-metrics
`network`	—	cloudflared(公開 Web 用 Tunnel)
`kube-system`	—	Traefik(k3s 同梱)・CoreDNS

管理元は `prod/test` が `2026kosensai` リポジトリの CI、それ以外が `manifest` リポジトリの CI。`web-static/prod/test` の IngressRoute は、過去年度アーカイブ(`web-static` namespace)の Service を cross-namespace で参照する。単一テナント運用のクラスタであるため、Traefik の `providers.kubernetesCRD.allowCrossNamespace: true` を有効にしている。

3 ネットワーク・セキュリティ

3.1 公開ポート

VPS の公開インターフェースに bind されるポートは 443/tcp のみ。80/tcp は Traefik の `web` entrypoint が ClusterIP のみで待ち受けており、ホストには一切 bind されない。

443/tcp へのアクセスは二重に制限している。

1. **VPS プロバイダのセキュリティグループ**: Cloudflare が公開している IPv4/IPv6 レンジ (`https://www.cloudflare.com/ips-v4`, `ips-v6`)のみ許可。
2. **ホスト側 nftables**(`k3s/cloudflare-ip-allowlist.sh`): 同じ Cloudflare IP レンジを、raw prerouting 段階でパケットの送信元インターフェース(`iifname`、デフォルトルートから動的に解決)と組み合わせて判定する。マッチしない場合は TCP ハンドシェイクにも TLS アラートにも至らずサイレントに drop する。systemd timer(`cloudflare-ip-allowlist.timer`)が日次でレンジを再取得し、`manifest` リポジトリへの push 時にも CI が都度再実行する。

3.2 TLS / mTLS

- **オリジンサーバ証明書**: Cloudflare Origin CA 証明書。SAN は `nara-kosensai.com` と `*.nara-kosensai.com` をカバー(有効期限 2041 年)。`2026kosensai` リポジトリの `production-vps` environment の GitHub Environment Secret(`CF_ORIGIN_CERT` / `CF_ORIGIN_KEY`)として保管し、CI が `prod` namespace に `cloudflare-origin-cert` (`kubernetes.io/tls`)Secret として投入する。

- **クライアント証明書検証**: Traefik の `TLSOption`(namespace: `network`、名前 `cloudflare-mtls`)が、Cloudflare が公開している Authenticated Origin Pulls 用 CA 証明書 (`k8s/base/network/cloudflare-aop-ca.yaml`、公開情報のため git 管理)に対して `RequireAndVerifyClientCert` を要求する。Free プランのため Cloudflare 管理の zone-level 証明書を使用(per-hostname のカスタムクライアント証明書は非対応)。

3.3 RBAC

Prometheus と kube-state-metrics は、それぞれ専用の ServiceAccount で動作し、read-only の `get/list/watch` のみを許可する ClusterRole を持つ(`k8s/base/monitoring/prometheus-rbac.yaml`、`kube-state-metrics.yaml`)。既定の ServiceAccount には何の権限も付与しない。

4 シークレット管理

Infisical(自己ホスト)は 2026-08-29 付けで完全に廃止した。現在の方針は次の通り。

対象	保管場所	理由
DB / pgAdmin / CORS 等(`app-env`)	`manifest` リポジトリ直下の `prod.env` / `staging.env`(平文コミット)	DB は ClusterIP 限定で外部到達不能
GHCR pull トークン	GitHub Environment Secret(`production-vps` / `test-vps`)	CI からのみ必要
Cloudflare Origin CA 証明書	GitHub Environment Secret(`production-vps`、環境限定)	本番専用の機微情報
Cloudflare Access クライアント資格情報	GitHub Actions Secret(管理用 SSH Tunnel 専用)	app-env とは無関係
Grafana 管理者パスワード等	k8s Secret(`grafana-secrets`)、ブートストラップ	gitignore 対象、循環依存を避けるため

対象	保管場所	理由
	時に手動投入	

manifest リポジトリの CI が prod.env / staging.env を VPS の/home/github/app-env/に 同期し、2026kosensai リポジトリの CI がそのパスを直接--from-env-file で参照する。このため manifest 側の CI が 2026kosensai 側より先に一度動いている必要がある。

5 CI/CD パイプライン

リポジトリ / ブランチ	処理内容
manifest → main	k3s ブートストラップマニフェスト再配置 → Cloudflare IP allowlist 再実行 → kubectl apply -k k8s/overlays/shared → 各 Deployment の rollout 確認
2026kosensai → prod	イメージ build/push → GHCR pull token・Origin CA 証明書を GitHub Secrets から取得 → kubectl apply -k infra/k8s/overlays/prod(backend/frontend/website 個別に path-filter 判定)→ rollout 確認
2026kosensai → test	同上(test namespace、pgAdmin 用 Cloudflare Access は別途手動設定)

いずれも GitHub Actions から VPS へは、管理用の Cloudflare Access Tunnel(cloudflared access tcp)経由で SSH 接続する。これは公開 Web 用の Tunnel(namespace network の cloudflared)とは 別物で、VPS 側からの outbound 接続であるため、§3 の nftables allowlist とは無関係に動作する。

6 監視

Prometheus(namespace monitoring)は次をスクレイプする。

- node-exporter: ノードの CPU/メモリ/ディスク
- traefik: Traefik 自身のメトリクス(現状 9100 番ポートが未公開のため要修正、既知の課題)
- kube-state-metrics: Pod/Deployment/StatefulSet のオブジェクト状態 (Running/Restart 回数/Ready レプリカ数)
- kubernetes-apiservers: kube-apiserver のリクエストレイテンシ・エラー率
- kubelet / kubernetes-cadvisor: ノード・Pod 単位のリソース使用量

いずれも kubernetes_sd_configs による in-cluster discovery と、ServiceAccount トークン (/var/run/secrets/kubernetes.io/serviceaccount/)による認証を使う標準的な構成。 Grafana の Prometheus/Loki データソースは自動プロビジョニングしておらず、初回のみ手動追加が必要 (http://prometheus:9090, http://loki:3100、いずれも monitoring namespace 内)。 ログは Grafana Alloy(DaemonSet、k3s/alloy-logging.yaml)が全 namespace の Pod ログを Loki へ出荷する。

7 障害と設計判断

7.1 nftables ルールのインターフェーススコープ(2026-08-29)

Origin Authentication 導入時に追加した nftables ルールが、当初 tcp dport 443 のみで 判定しており、送信元インターフェースを指定していなかった。クラスタ内部には kubernetes.default.svc(ClusterIP、

10.43.0.1:443)を含む複数の ClusterIP が たまたま 443 番を使っており、このルールは Pod→APIサーバー間の内部通信まで 巻き込んで drop してしまった。

結果として helm-install-traefik ジョブ(Traefik の新しいエントリーポイント設定を 適用するための Helm インストール)が API サーバーに到達できず、インストール失敗→アンインストール→再インストールのクラッシュループに陥り、 Traefik 自体が消滅。本番環境が約 1 時間ダウンした(Cloudflare エラー 521/526)。

対処: ルールを iifname(ノードの実際の外部 NIC。ハードコードではなく ip route show default から都度動的に解決)でスコープした。クラスタ内部の トラフィックは定義上この物理インターフェースを通らないため、どの ClusterIP が 443 番を使っていても衝突しない。

同時に、Traefik のコンテナポート・Service 公開ポートでも類似の衝突が発生していた (websecure-mtls を chart のデフォルト websecure と同じ 8443 番/443 番で定義していたため)。コンテナポートは 8444 番、Service 公開は expose.default: false(hostPort のみ、Service 経由の公開はそもそも不要)に変更して解消した。

8 リファレンス

項目	値
Kubernetes ディストリビューション	k3s(シングルノード)
Ingress コントローラ	Traefik 3.7(k3s 同梱 Helm チャート)
CNI	flannel(k3s 標準)
ストレージ	local-path(既定)、local-path-retain(データ保護が必要な PVC 用)
公開ポート	443/tcp のみ(80/tcp は ClusterIP 限定で非公開)
Traefik websecure-mtls	containerPort 8444, hostPort 443, expose.default: false
Traefik web	containerPort 8000 相当(chart 既定), ClusterIP:80 のみ
管理用 SSH	Cloudflare Access Tunnel(公開 Web 用 Tunnel とは別物、outbound-only)